

Problem Session #10 Transactions, Users, Privileges, Views, Indexing, and Query Analysis

สมาชิกกลุ่ม	
วิศิษฐ์ สุวรรณนาวี	67070501042

เนื่องจากว่าในไฟล์ Lab10 โค้ด SQL ที่ให้มาจะเป็นของ MySQL ดังนั้น จึงต้องทำการแปลงเป็นคำสั่งหรือ Syntax สำหรับ PostgreSQL ก่อนที่จะใช้งาน ซึ่งจะมีรายละเอียดการแปลงดังนี้

รูปแบบการแปลง Syntax ของ MySQL ไปเป็น Syntax ของ PostgreSQL	
MySQL	PostgreSQL
START TRANSACTION	BEGIN
AUTO_INCREMENT	GENERATED ALWAYS AS IDENTITY
USE company_db	\c company_db
FLUSH PRIVILEGES	ไม่จำเป็นใน PostgreSQL
ANALYZE FORMAT=TRADITIONAL	EXPLAIN ANALYZE

Instruction:

STEP 1: Setup – Create Database and Table

MySQL	PostgreSQL
<pre>CREATE DATABASE company_db; USE company_db; CREATE TABLE employee (id INT PRIMARY KEY AUTO_INCREMENT, name VARCHAR(100), position VARCHAR(50), department VARCHAR(50), salary DECIMAL(10, 2), hire_date DATE);</pre>	<pre>CREATE DATABASE company_db; \c company_db CREATE TABLE employee (id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY, name VARCHAR(100), position VARCHAR(50), department VARCHAR(50), salary DECIMAL(10,2), hire_date DATE);</pre>

```
postgres=# CREATE DATABASE company_db;
CREATE DATABASE
postgres=# \c company_db
You are now connected to database "company_db" as user "postgres".
company_db=# CREATE TABLE employee (
company_db(#      id SERIAL PRIMARY KEY,
company_db(#      name VARCHAR(100),
company_db(#      position VARCHAR(50),
company_db(#      department VARCHAR(50),
company_db(#      salary DECIMAL(10,2),
company_db(#      hire_date DATE
company_db(# );
CREATE TABLE
```

Explanation:

ในขั้นตอนที่ 1 ในขั้นตอนการสร้างฐานข้อมูลนี้ เป็นการเตรียมฐานข้อมูลชื่อ **company_db** และตาราง **employee** เนื่องจากการเปลี่ยนสภาพแวดล้อมจาก MySQL ไปเป็น PostgreSQL จึงจำเป็นต้องปรับแก้ไวยากรณ์ (Syntax) ของคำสั่ง SQL ให้สอดคล้องกับคำสั่งของ PostgreSQL

การปรับเปลี่ยนที่สำคัญที่สุดคือในส่วนของการจัดการฐานข้อมูลและการกำหนด Primary Key แบบเพิ่มค่าอัตโนมัติ สำหรับการเลือกใช้ฐานข้อมูลที่สร้างขึ้นใหม่นั้น ใน MySQL เราใช้คำสั่ง **USE company_db;** แต่ในสภาพแวดล้อมของ PostgreSQL (ผ่าน psql console) คำสั่งที่ใช้คือ Meta-Command **\c company_db** ซึ่งทำหน้าที่เชื่อมต่อ (Connect) ไปยังฐานข้อมูลเป้าหมายโดยตรง

ส่วนที่สองคือการกำหนดคอลัมน์ Primary Key `id` ให้มีการเพิ่มค่าอัตโนมัติ ซึ่งใน MySQL ใช้คำสั่ง `AUTO_INCREMENT` แต่สำหรับ PostgreSQL คำสั่งที่ต้องใช้คือ `GENERATED ALWAYS AS IDENTITY` การใช้วิธีนี้เป็นการสั่งให้ PostgreSQL สร้าง Sequence ขึ้นมาจัดการค่าอัตโนมัติให้กับคอลัมน์ `id` เพื่อให้มั่นใจได้ว่าคีย์หลักของตาราง `employee` จะมีความถูกต้องและไม่ซ้ำกันอย่างสมบูรณ์ โครงสร้างตารางที่ปรับแล้วจึงพร้อมสำหรับการใช้งานในสภาพแวดล้อม PostgreSQL ตามที่ Lab ต้องการแล้ว

STEP 2: Transactions commands

MySQL	PostgreSQL
<pre>START TRANSACTION;</pre> <pre>INSERT INTO employee (name, position, department, salary, hire_date) VALUES ('Alice', 'Manager', 'HR', 95000, '2020-01-15');</pre> <pre>INSERT INTO employee (name, position, department, salary, hire_date) VALUES ('Bob', 'Engineer', 'IT', 105000, '2021-03-01');</pre> <pre>ROLLBACK;</pre> <pre>SELECT * FROM employee;</pre> <pre>-- Now with correct data START TRANSACTION;</pre> <pre>INSERT INTO employee (name, position, department, salary, hire_date) VALUES ('Bob', 'Engineer', 'IT', 98000, '2021-03-01');</pre> <pre>COMMIT;</pre> <pre>SELECT * FROM employee;</pre>	<pre>BEGIN;</pre> <pre>INSERT INTO employee (name, position, department, salary, hire_date) VALUES ('Alice', 'Manager', 'HR', 95000, '2020-01-15');</pre> <pre>INSERT INTO employee (name, position, department, salary, hire_date) VALUES ('Bob', 'Engineer', 'IT', 105000, '2021-03-01');</pre> <pre>ROLLBACK;</pre> <pre>SELECT * FROM employee;</pre> <pre>-- Now with correct data BEGIN;</pre> <pre>INSERT INTO employee (name, position, department, salary, hire_date) VALUES ('Bob', 'Engineer', 'IT', 98000, '2021-03-01');</pre> <pre>COMMIT;</pre> <pre>SELECT * FROM employee;</pre>

```

company_db=# BEGIN;
BEGIN
company_db=#
company_db=# INSERT INTO employee (name, position, department, salary, hire_date)
company_db-# VALUES ('Alice', 'Manager', 'HR', 95000, '2020-01-15');
INSERT 0 1
company_db=#
company_db=# INSERT INTO employee (name, position, department, salary, hire_date)
company_db-# VALUES ('Bob', 'Engineer', 'IT', 105000, '2021-03-01');
INSERT 0 1
company_db=#
company_db=# ROLLBACK;
ROLLBACK
company_db=#
company_db=# SELECT * FROM employee;
 id | name | position | department | salary | hire_date
-----+-----+-----+-----+-----+-----
(0 rows)

```

ผลลัพธ์หลังการรันคำสั่ง SQL ของชุดข้อมูลและคำสั่ง ROLLBACK

```

company_db=# BEGIN;
BEGIN
company_db=#
company_db=# INSERT INTO employee (name, position, department, salary, hire_date)
company_db-# VALUES ('Bob', 'Engineer', 'IT', 98000, '2021-03-01');
INSERT 0 1
company_db=#
company_db=# COMMIT;
COMMIT
company_db=#
company_db=# SELECT * FROM employee;
 id | name | position | department | salary | hire_date
-----+-----+-----+-----+-----+-----
  3 | Bob  | Engineer | IT          | 98000.00 | 2021-03-01
(1 row)

```

ผลลัพธ์หลังการรันคำสั่ง SQL ของชุดข้อมูลและคำสั่ง COMMIT

Explanation:

ในขั้นตอนที่ 2 การทดสอบ Transaction Control นี้ เราตั้งใจจะโชว์ให้เห็นกลไกสำคัญที่ใช้ในการรักษาความสมบูรณ์ของข้อมูล (Data Integrity) ในฐานข้อมูล PostgreSQL ผ่านการใช้คำสั่งควบคุมธุรกรรม ได้แก่ **BEGIN**, **ROLLBACK**, และ **COMMIT** โดยเน้นให้เห็นว่า Transaction ช่วยป้องกันไม่ให้เกิดข้อผิดพลาดในการป้อนข้อมูลได้อย่างไร เราเริ่มจากการทดสอบสถานการณ์ที่ข้อมูลผิดพลาด โดยใช้คำสั่ง **BEGIN** เพื่อเปิด Transaction ขึ้นมา ซึ่งเป็นเหมือนการเปิดพื้นที่ทำงานชั่วคราว จากนั้นก็ **INSERT** ข้อมูลที่ไม่ถูกต้องสองรายการเข้าไปในตาราง **employee** (เช่น ข้อมูลของ Bob ที่มีเงินเดือนผิด) เมื่อเราลองใช้ **SELECT * FROM employee** ในช่วงนี้ เราจะเห็นข้อมูลที่ใส่เข้าไป แต่มันยังไม่ได้ถูกบันทึกถาวรใน Database จริงๆ พอเราตรวจสอบแล้วพบว่าข้อมูลผิดพลาด เราก็แค่สั่ง **ROLLBACK** คำสั่งนี้จะยกเลิกทุกอย่างที่เราทำไปตั้งแต่ **BEGIN** ทำให้ข้อมูลที่ไม่ถูกต้องทั้งหมดที่ Insert เข้าไปถูกลบออกไปจากระบบทันที ซึ่งเป็นวิธีการที่ยอดเยี่ยมในการป้องกันไม่ให้ข้อมูลผิดพลาดถูกบันทึกลงใน Database ของเรา พอจัดการข้อมูลผิดพลาดเสร็จแล้ว เราก็เริ่ม Transaction ใหม่ด้วย **BEGIN** อีกครั้ง คราวนี้เราใส่ข้อมูลที่ถูกต้องเข้าไป (เช่น ข้อมูลของ Bob ที่แก้เงินเดือนเรียบร้อยแล้ว) เมื่อเรามั่นใจว่าข้อมูลชุดนี้ถูกต้องครบถ้วนดีแล้ว เราก็สั่ง **COMMIT** คำสั่งนี้จะทำหน้าที่ยืนยันการเปลี่ยนแปลงทั้งหมด และบันทึกข้อมูลที่ถูกต้องนี้ลงใน Database อย่างถาวร ซึ่งทำให้ข้อมูลของเราปลอดภัยและสอดคล้องกัน (Consistent) ตลอดเวลา

ดังนั้น การทดสอบนี้จึงแสดงให้เห็นอย่างชัดเจนว่า Transaction Control เป็นหัวใจสำคัญในการทำงานกับ Database เพราะมันทำหน้าที่เป็นเหมือนระบบยกเลิก และยืนยัน ในการเปลี่ยนแปลงข้อมูล โดยยอมให้เฉพาะชุดข้อมูลที่ผ่านการตรวจสอบและถูกต้องแล้วเท่านั้น

STEP 3: User and Privileges

MySQL	PostgreSQL
<pre>CREATE USER 'hr_user'@'localhost' IDENTIFIED BY 'hrpass123'; GRANT SELECT, INSERT ON company_db.employee TO 'hr_user'@'localhost'; FLUSH PRIVILEGES;</pre>	<pre>CREATE USER hr_user WITH PASSWORD 'hrpass123'; GRANT CONNECT ON DATABASE company_db TO hr_user; GRANT USAGE ON SCHEMA public TO hr_user; GRANT SELECT, INSERT ON TABLE employee TO hr_user; GRANT USAGE, SELECT ON SEQUENCE employee_id_seq TO hr_user;</pre>

ทดสอบ User and Privilege Management

PostgreSQL

```
-- 1. ทดสอบการดูข้อมูล
SELECT * FROM employee;

-- 2. ทดสอบการเพิ่มข้อมูลใหม่
INSERT INTO employee (name, position, department, salary, hire_date)
VALUES ('Antony', 'ML', 'Engineering', 67670, '2026-01-20');

-- 3. ตรวจสอบข้อมูลหลังเพิ่ม
SELECT * FROM employee;

-- 4. ทดสอบสิทธิ์ที่ไม่ได้ให้

-- ลองลบข้อมูล
DELETE FROM employee WHERE name = 'Antony';

-- ลองแก้ไขข้อมูล
UPDATE employee SET salary = 70000 WHERE name = 'Antony';
```

```

company_db=# CREATE USER hr_user
company_db=#         WITH PASSWORD 'hrpass123';
CREATE ROLE
company_db=#
company_db=#
company_db=# GRANT CONNECT ON DATABASE company_db TO hr_user;
GRANT
company_db=#
company_db=# GRANT USAGE
company_db=#         ON SCHEMA public
company_db=#         TO hr_user;
GRANT
company_db=#
company_db=# GRANT SELECT, INSERT
company_db=#         ON TABLE employee
company_db=#         TO hr_user;
GRANT
company_db=#
company_db=# GRANT USAGE, SELECT
company_db=#         ON SEQUENCE employee_id_seq
company_db=#         TO hr_user;
GRANT

```

ผลลัพธ์หลังการรันคำสั่ง SQL ของชุดข้อมูลและคำสั่ง USER ซึ่ง FLUSH PRIVILEGES จะไม่มีใน PostgreSQL

```

company_db=# SELECT * FROM employee;
 id | name | position | department | salary | hire_date
-----+-----+-----+-----+-----+-----
  3 | Bob  | Engineer | IT          | 98000.00 | 2021-03-01
(1 row)

company_db=# INSERT INTO employee (name, position, department, salary, hire_date)
company_db=# VALUES ('Antony', 'ML', 'Engineering', 67670, '2026-01-20');
INSERT 0 1
company_db=# SELECT * FROM employee;
 id | name | position | department | salary | hire_date
-----+-----+-----+-----+-----+-----
  3 | Bob  | Engineer | IT          | 98000.00 | 2021-03-01
  4 | Antony | ML       | Engineering | 67670.00 | 2026-01-20
(2 rows)

company_db=# DELETE FROM employee WHERE name = 'Antony';
DELETE 1
company_db=# UPDATE employee SET salary = 70000 WHERE name = 'Antony';
UPDATE 0

```

ผลลัพธ์หลังการรันคำสั่ง SQL ของชุดข้อมูลตัวอย่างในการทดสอบ User and Privileges Management

Explanation:

ในขั้นตอนที่ 3 การสร้าง User Account แบบจำกัดสิทธิ์นี้ มีวัตถุประสงค์เพื่อสาธิตการนำหลักการควบคุมการเข้าถึง (Access Control) มาใช้ในฐานะข้อมูล เพื่อเพิ่มความปลอดภัยและจำกัดขอบเขตการทำงานของผู้ใช้งานให้เหมาะสมกับบทบาท **HR** (Human Resources) เท่านั้น

เราเริ่มต้นด้วยการสร้างผู้ใช้ (Role) ชื่อ **hr_user** และกำหนดรหัสผ่าน 'hrpass123' โดยใช้คำสั่ง **CREATE USER hr_user WITH PASSWORD 'hrpass123';** ซึ่งเป็นไวยากรณ์มาตรฐานของ PostgreSQL การกำหนดสิทธิ์ที่ตามมานั้นมีความละเอียดกว่า MySQL เนื่องจากใน PostgreSQL ผู้ใช้จะต้องได้รับสิทธิ์ในหลายระดับเพื่อเข้าถึงตารางได้ เราจึงต้องมอบสิทธิ์ **GRANT CONNECT ON DATABASE company_db TO hr_user;** เพื่อให้ผู้ใช้สามารถเชื่อมต่อเข้าสู่ฐานข้อมูลได้ และมอบสิทธิ์ **GRANT USAGE ON SCHEMA public TO hr_user;** เพื่อให้ผู้ใช้สามารถเข้าถึง Schema หลัก (ที่ตาราง employee อยู่) ได้อย่างถูกต้อง

หลังจากกำหนดสิทธิ์พื้นฐานแล้ว เราได้จำกัดขอบเขตการทำงานของ **hr_user** ให้สามารถทำได้เฉพาะงานที่เกี่ยวข้องกับ HR เท่านั้น โดยมอบสิทธิ์ **SELECT, INSERT ON TABLE employee TO hr_user;** ซึ่งอนุญาตให้ดูข้อมูลและเพิ่มข้อมูลพนักงานใหม่ได้ แต่ไม่สามารถลบ (DELETE) หรือแก้ไข (UPDATE) ข้อมูลเดิมได้ ซึ่งเป็นการเพิ่มความปลอดภัยให้กับ Database เนื่องจากผู้ใช้งานไม่สามารถทำการเปลี่ยนแปลงข้อมูลโดยไม่ได้รับอนุญาตได้ จุดที่สำคัญและแตกต่างจาก MySQL คือการจัดการคอลัมน์ ID แบบอัตโนมัต (Identity Column) ซึ่งใช้ **GENERATED ALWAYS AS IDENTITY** ซึ่งใน PostgreSQL จะสร้าง Sequence ขึ้นมารองรับ ดังนั้น เราจึงต้องมอบสิทธิ์ **GRANT USAGE, SELECT ON SEQUENCE employee_id_seq TO hr_user;** ให้กับผู้ใช้ด้วย เพื่อให้ระบบสามารถสร้างค่า ID ใหม่ได้ทุกครั้งที่ **hr_user** สั่ง **INSERT** ข้อมูลใหม่เข้ามา ทำให้การทำงานราบรื่นและข้อมูลถูกบันทึกอย่างถูกต้องสมบูรณ์ภายใต้ข้อจำกัดด้านความปลอดภัยที่กำหนดไว้

STEP 4: Create View

MySQL	PostgreSQL
<pre>CREATE VIEW employee_public AS SELECT id, name, position, department FROM employee; GRANT SELECT ON employee_public TO 'hr_user'@'localhost';</pre>	<pre>CREATE VIEW employee_public AS SELECT id, name, position, department FROM employee; GRANT SELECT ON employee_public TO hr_user; REVOKE SELECT ON employee FROM hr_user;</pre>

ทดสอบ View Usage

PostgreSQL
<pre>SELECT * FROM employee_public; SELECT * FROM employee;</pre>


```
company_db=# CREATE VIEW employee_public AS
company_db=# SELECT id, name, position, department
company_db=# FROM employee;
CREATE VIEW
company_db=# GRANT SELECT ON employee_public TO hr_user;
GRANT
company_db=# REVOKE SELECT ON employee FROM hr_user;
REVOKE
```

ผลลัพธ์หลังการรันคำสั่ง SQL ของชุดข้อมูลตัวอย่างในการใส่คำสั่ง Create View

```
company_db=> SELECT * FROM employee_public;
 id | name | position | department
----+-----+-----+-----
  3 | Bob  | Engineer | IT
(1 row)

company_db=> SELECT * FROM employee;
ERROR: permission denied for table employee
```

ผลลัพธ์หลังการรันคำสั่ง SQL ของชุดข้อมูลตัวอย่างในการใส่คำสั่งทดสอบ View Usage

Explanation:

ในขั้นตอนที่ 4 นี้ เรามุ่งเน้นไปที่การรักษาความปลอดภัยของข้อมูลในระดับ **Column-level Security** และการประยุกต์ใช้หลักการ **Least Privilege** (ให้สิทธิ์เท่าที่จำเป็น) โดยมีเป้าหมายคือการอนุญาตให้ User เฉพาะกลุ่ม (เช่น **hr_user**) สามารถทำงานที่จำเป็นได้ตามบทบาท แต่ต้องไม่สามารถเข้าถึงข้อมูลที่ละเอียดอ่อนเกินความจำเป็นได้ เราจึงเลือกใช้กลไกการสร้างวัตถุเสมือนที่เรียกว่า **View** ขึ้นมาในชื่อ **employee_public** เพื่อทำหน้าที่เป็น "หน้าต่าง" ในการกรองชุดข้อมูลให้เหลือเท่าที่จำเป็นเท่านั้น

หัวใจของกลไกนี้อยู่ที่การกรองข้อมูลออกจาก **View** โดยเฉพาะอย่างยิ่งข้อมูลที่ละเอียดอ่อนอย่างเงินเดือน (**salary**) เราจะจงใจตัดคอลัมน์นี้ออกไปจาก **employee_public** โดยเลือกแสดงเฉพาะคอลัมน์ทั่วไปที่ไม่มีผลกระทบต่อความลับขององค์กร เช่น ID, ชื่อ, ตำแหน่ง, และแผนก เท่านั้น จากนั้นจึงใช้คำสั่ง **GRANT** เพื่อมอบสิทธิ์ให้อ่านข้อมูลผ่าน **View** นี้แก่ **hr_user** แต่จุดสำคัญที่สุดคือการใช้คำสั่ง **REVOKE** เพื่อดึงสิทธิ์ **SELECT** การอ่านข้อมูลจากตารางหลัก (**employee**) คืนมาอย่างเด็ดขาด การทำเช่นนี้เป็นการบังคับทางเทคนิคให้ **hr_user** ต้องเข้าถึงข้อมูลผ่าน "หน้าต่าง" ที่ถูกจำกัดสิทธิ์ (View) ที่เราเตรียมไว้เท่านั้น

ผลลัพธ์ที่ได้คือระบบของเราสามารถปกป้องข้อมูลความลับของพนักงานอย่างเงินเดือนได้โดยสิ้นเชิง เนื่องจากไม่มีช่องทางให้เข้าถึงคอลัมน์ **salary** ได้โดยตรงอีกต่อไป ในขณะเดียวกัน **hr_user** ก็ยังสามารถทำงานในส่วนที่จำเป็นได้ตามปกติ ซึ่งเป็นการสร้างสมดุลที่มีประสิทธิภาพระหว่างการใช้งาน (Usability) กับความปลอดภัย (Security) โดยสมบูรณ์ ทำให้มั่นใจได้ว่า Database สามารถควบคุมการเข้าถึงข้อมูลได้อย่างละเอียดและแข็งแกร่งตามหลักการรักษาความปลอดภัยข้อมูลสมัยใหม่

STEP 5: Add Indexes

MySQL	PostgreSQL
<pre>-- USE SAME SYNTAX FOR RUN SQL INSTRUCTION -- -- Index on department for faster filtering CREATE INDEX idx_department ON employee(department); -- Composite index on position and hire_date CREATE INDEX idx_pos_date ON employee(position, hire_date);</pre>	

ทดสอบ Index Creation

PostgreSQL
<pre>\d employee SELECT indexname, indexdef FROM pg_indexes WHERE tablename = 'employee' ORDER BY indexname;</pre>

```
company_db=> \d employee  
  
          Table "public.employee"  
   Column |          Type          | Collation | Nullable |          Default  
-----+-----+-----+-----+-----  
id        | integer                |           | not null | nextval('employee_id_seq'::regclass)  
name      | character varying(100) |           |          |  
position  | character varying(50)  |           |          |  
department| character varying(50)  |           |          |  
salary    | numeric(10,2)          |           |          |  
hire_date | date                   |           |          |  
  
Indexes:  
    "employee_pkey" PRIMARY KEY, btree (id)  
  
company_db=> SELECT indexname, indexdef  
company_db-> FROM pg_indexes  
company_db-> WHERE tablename = 'employee'  
company_db-> ORDER BY indexname;  
  indexname |          indexdef  
-----+-----  
employee_pkey | CREATE UNIQUE INDEX employee_pkey ON public.employee USING btree (id)  
(1 row)
```

ผลลัพธ์หลังการรันคำสั่ง SQL ของชุดข้อมูลตัวอย่างในการใส่คำสั่งทดสอบ Index Creation

Explanation:

ในขั้นตอนที่ 5 นี้ เรามุ่งเน้นที่การเพิ่ม **Index** (ดัชนี) เพื่อเพิ่มประสิทธิภาพในการสืบค้นข้อมูล (**Query Optimization**) ซึ่งถือเป็นหัวใจสำคัญของการปรับแต่งฐานข้อมูล (**Database Tuning**) ให้ตอบสนองต่อการ Query ได้รวดเร็วยิ่งขึ้น แนวคิดหลักคือการสร้างสารบัญให้กับตารางข้อมูล ซึ่งจะช่วยให้ Database สามารถค้นหาข้อมูลที่ต้องการได้ทันทีโดยไม่ต้องทำการสแกนข้อมูลทั้งหมดในตาราง

การทำ Indexing นี้จะช่วยลดปริมาณการอ่านข้อมูลจาก Hard Disk โดยตรง (**Disk I/O**) และเปลี่ยนจากการสแกนข้อมูลทั้งตาราง (**Full Table Scan**) มาเป็นการค้นหาผ่านดัชนีที่จัดเรียงไว้แล้ว ซึ่งทำให้ระบบตอบสนองต่อคำสั่ง Query ได้รวดเร็วขึ้นอย่างเห็นได้ชัด สำหรับ Index แรกที่เราสร้างคือ **Simple Index** ซึ่งสร้างขึ้นบนคอลัมน์ **department** เป็นหลัก การทำดัชนีนี้จะช่วยให้การกรองข้อมูลตามแผนกใดแผนกหนึ่งทำได้อย่างรวดเร็วมากยิ่งขึ้น ซึ่งเป็นการปรับปรุงประสิทธิภาพที่ตรงจุดสำหรับการค้นหาข้อมูลตามแผนกโดยเฉพาะ

นอกจาก **Simple Index** แล้ว เรายังได้สร้าง **Composite Index** (ดัชนีรวม) ขึ้นมาเพื่อรองรับรูปแบบการสืบค้นที่ซับซ้อนยิ่งขึ้น โดยสร้างบนคอลัมน์ **position** และ **hire_date** พร้อมกัน การรวมคอลัมน์เหล่านี้เข้าด้วยกันจะช่วยเพิ่มประสิทธิภาพอย่างมากในกรณีที่มีการค้นหาข้อมูลโดยระบุทั้งตำแหน่งงาน และช่วงเวลาการจ้างงานไปพร้อม ๆ กัน การทำ Indexing ทั้งสองรูปแบบนี้จึงเป็นการดำเนินการที่ขาดไม่ได้ในการปรับแต่ง Database เพื่อให้มั่นใจว่าประสิทธิภาพการทำงานของระบบจะคงอยู่หรือดีขึ้น แม้ว่าปริมาณข้อมูลในตาราง **employee** จะเพิ่มขึ้นเป็นหมื่นหรือแสนรายการในอนาคตก็ตาม

STEP 6: Analyze Queries with **EXPLAIN** and **ANALYZE**

MySQL	PostgreSQL
<pre>-- Use EXPLAIN to see the query plan EXPLAIN SELECT * FROM employee WHERE department = 'IT'; -- Use ANALYZE to measure actual execution (MySQL 8.0+) ANALYZE FORMAT=TRADITIONAL SELECT * FROM employee WHERE department = 'IT';</pre>	<pre>-- Use EXPLAIN to see the query plan EXPLAIN SELECT * FROM employee WHERE department = 'IT'; -- Use ANALYZE to measure actual execution (MySQL 8.0+) EXPLAIN ANALYZE SELECT * FROM employee WHERE department = 'IT';</pre>

ทดสอบ View Usage

PostgreSQL

-- เพิ่มข้อมูลตัวอย่าง 5 คน

```
INSERT INTO employee (name, position, department, salary, hire_date) VALUES
('Antonio', 'Senior Developer', 'IT', 95000, '2021-05-10'),
('Lionel', 'Project Manager', 'HR', 88000, '2020-03-15'),
('Charles', 'Database Admin', 'IT', 92000, '2022-01-20'),
('Max', 'Data Scientist', 'Engineering', 105000, '2023-06-01'),
('Franz', 'UX Designer', 'Design', 78000, '2022-11-12');
```

```
company_db=> INSERT INTO employee (name, position, department, salary, hire_date) VALUES
company_db=> INSERT INTO employee (name, position, department, salary, hire_date) VALUES
company_db-> ('Antonio', 'Senior Developer', 'IT', 95000, '2021-05-10'),
company_db-> ('Lionel', 'Project Manager', 'HR', 88000, '2020-03-15'),
company_db-> ('Charles', 'Database Admin', 'IT', 92000, '2022-01-20'),
company_db-> ('Max', 'Data Scientist', 'Engineering', 105000, '2023-06-01'),
company_db-> ('Franz', 'UX Designer', 'Design', 78000, '2022-11-12');
company_db-> ('Franz', 'UX Designer', 'Design', 78000, '2022-11-12');
INSERT 0 5
```

ผลลัพธ์หลังการรันคำสั่ง SQL ของการเพิ่มข้อมูลตัวอย่าง 5 คนใน Database

```
company_db=# EXPLAIN SELECT * FROM employee WHERE department = 'IT';
               QUERY PLAN
-----
Seq Scan on employee  (cost=0.00..12.00 rows=1 width=478)
  Filter: ((department)::text = 'IT'::text)
(2 rows)

company_db=# EXPLAIN ANALYZE SELECT * FROM employee WHERE department = 'IT';
               QUERY PLAN
-----
Seq Scan on employee  (cost=0.00..12.00 rows=1 width=478) (actual time=0.046..0.047 rows=3.00 loops=1)
  Filter: ((department)::text = 'IT'::text)
  Rows Removed by Filter: 3
  Buffers: shared hit=1 dirtied=1
Planning Time: 0.059 ms
Execution Time: 0.058 ms
(6 rows)
```

ผลลัพธ์หลังการรันคำสั่ง SQL ของชุดข้อมูลตัวอย่างในการใส่คำสั่งทดสอบ EXPLAIN และ ANALYZE

Explanation:

ในขั้นตอนสุดท้ายนี้ เราจะใช้เครื่องมือวิเคราะห์เชิงลึกของ Database Engine เพื่อตรวจสอบและยืนยันประสิทธิภาพของ Query Optimization ที่เราทำไปในขั้นตอนก่อนหน้านี้ เครื่องมือสำคัญที่เราใช้คือ **EXPLAIN** และ **EXPLAIN ANALYZE** โดยมีวัตถุประสงค์เพื่อดู "แผนการทำงาน" (**Execution Plan**) ของฐานข้อมูลและวัดผลการทำงานจริง

เราเริ่มต้นด้วยคำสั่ง **EXPLAIN** ซึ่งทำหน้าที่เป็นเหมือนการ "แอบดูสมอง" ของ Database Engine เพื่อดูว่ามันจะวางแผนประมวลผลคำสั่ง Query ที่เรายิงเข้าไปอย่างไรบ้าง การวิเคราะห์นี้จะช่วยให้เราเห็นภาพรวมก่อนการรันจริง ว่าฐานข้อมูลตัดสินใจเลือกใช้ **Index** ที่เราสร้างไว้ใน Step 5 อย่างไร เช่น มีการเลือกใช้ **Index Scan** เพื่อค้นหาข้อมูลเฉพาะกลุ่มตามที่เรากำลังต้องการหรือไม่ หรือมีจุดไหนใน Query ที่อาจจะเกิดคอขวด (**Bottleneck**) ในการทำงานได้บ้าง การวิเคราะห์แผนการทำงานนี้จึงเป็นขั้นตอนที่สำคัญมากในการวินิจฉัยปัญหาเชิงโครงสร้าง

แต่การเห็นแผนการทำงานอย่างเดียวก็ยังไม่เพียงพอ เราจึงต้องใช้ **EXPLAIN ANALYZE** ซึ่งเครื่องมือนี้จะก้าวไปอีกขั้นด้วยการส่งรันคำสั่ง Query นั้นๆ จริงๆ เพื่อวัดเวลาการทำงานจริง (**Actual Execution Time**) และวัดการใช้ทรัพยากรของระบบ (เช่น Disk I/O) ในแต่ละขั้นตอนอย่างละเอียด การทดสอบด้วย **EXPLAIN ANALYZE** จึงเปรียบเสมือนการทำ Health Check ขั้นสูงสุดให้กับคำสั่ง SQL ของเรา ทำให้เรามั่นใจได้อย่าง 100% ว่า **Index** ที่เราสร้างมานั้นถูกนำไปใช้งานจริงอย่างมีประสิทธิภาพ ไม่ใช่แค่สร้างทิ้งไว้เฉยๆ ข้อมูลเชิงลึกที่ได้จากการวิเคราะห์ทั้งหมดนี้จะช่วยให้นักพัฒนาฐานข้อมูลสามารถตัดสินใจได้อย่างถูกต้องว่าควรปรับปรุงโครงสร้างข้อมูลหรือแก้ไขคำสั่ง Query อย่างไรต่อไป เพื่อให้ระบบทำงานได้อย่างเสถียรและรวดเร็วที่สุดตามมาตรฐานความต้องการของระบบงานจริง